MEMORANDUM

To:	Professor Brendon Anderson, Mechanical Engineering, California Polytechnic State University, bga@calpoly.edu
From:	Gabriel Choi, Carmen Alaichamy
Date:	April 21, 2026
Subject:	Lab 1A: Open-Loop Simulation

Introduction

The objective of this lab was to simulate and control the vertical landing of a rocket booster modeled as a pendulum. A proportional (P) controller and a proportional-derivative (PD) controller were implemented in a Python-based simulation environment. The effects of controller gains on system response, including overshoot and settling time, were analyzed. The report presents the implementation process, simulation results, controller tuning, and the influence of proportional and derivative gains on system performance.

Methodology

The rocket booster system was modeled as a simplified pendulum with angular position and velocity as states. A Jupyter Notebook simulation environment was used to implement the control system.

A proportional controller was first implemented using the control law u = Kp * e, where the error e is the difference between the desired angle and the measured angle. The proportional gain Kp was varied to observe its effect on system response.

Next, a proportional-derivative (PD) controller was implemented using the control law $u = K_p * e + K_d * \dot{e}$, where edot represents the derivative of the error. The derivative gain Kd was varied while holding Kp constant.

Simulation results were recorded for different gain values, including peak angle, final steady-state angle, percent overshoot, and settling time. Settling time was estimated from the response plots based on when the system remained within a 2% band of the final value.

Finally, controller gains were tuned to meet the design requirements of less than 5% overshoot and a settling time of less than 5 seconds.

Results

First, the results of the Python warm-up assignment is documented in Appendix A, where a function “Fibonacci” accepts one input *n*, and outputs the first *n*-th Fibonacci number and their sum.

Next, the rocket boost system results are shown below. The following Figures 1-3 plot the response of angle, velocity and displacement v.s. time of the tuned PD controller.

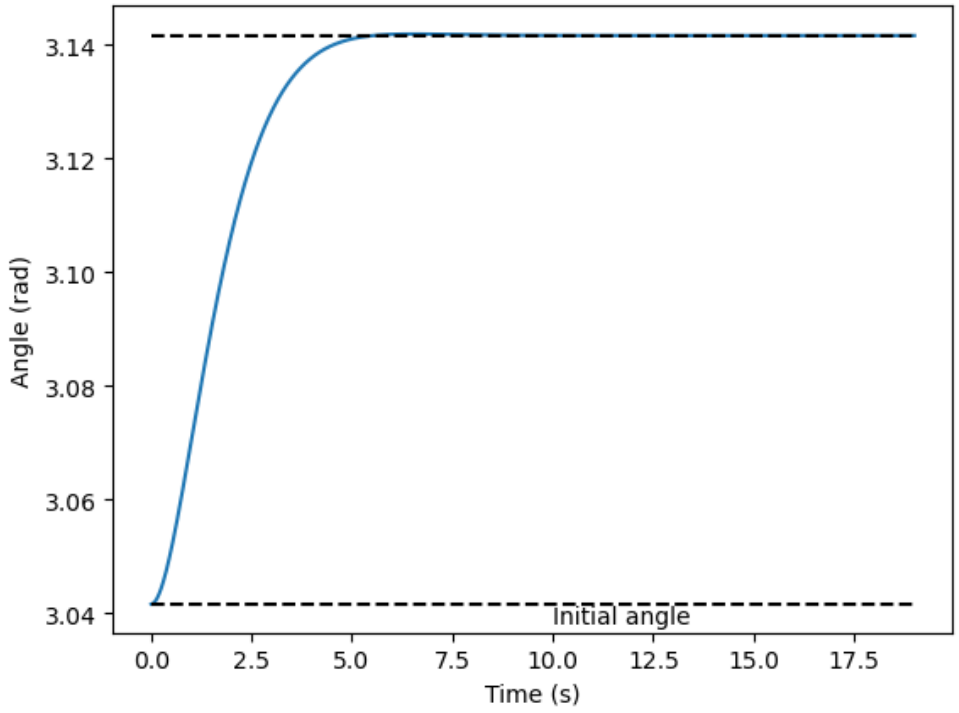


Figure 1. Angle(θ) Vs. Time (s) Model Response

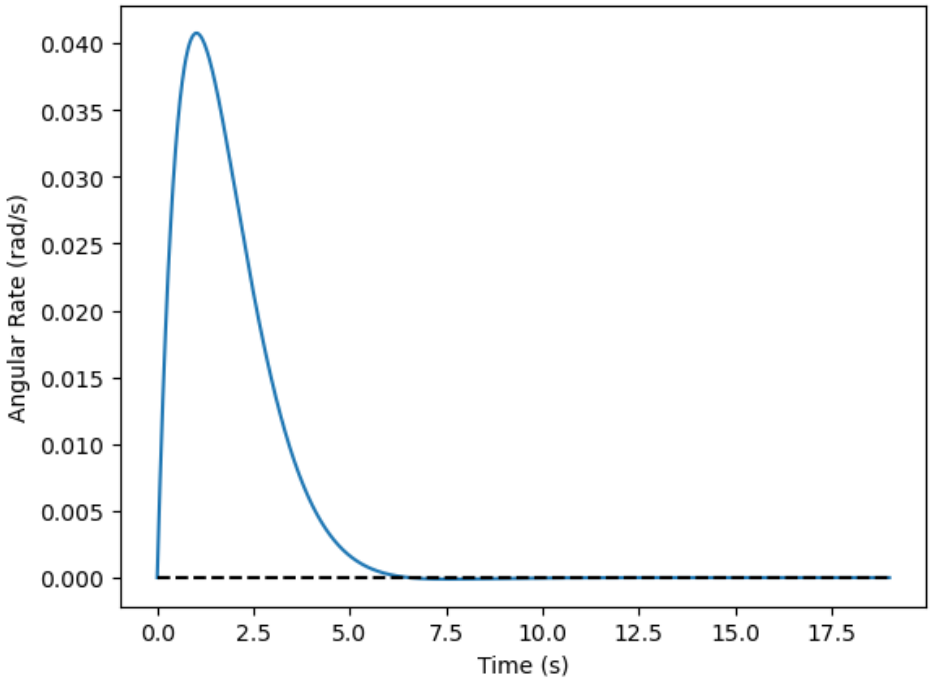


Figure 2. Angular Velocity($\dot{\theta}$) Vs. Time (s) Model Response

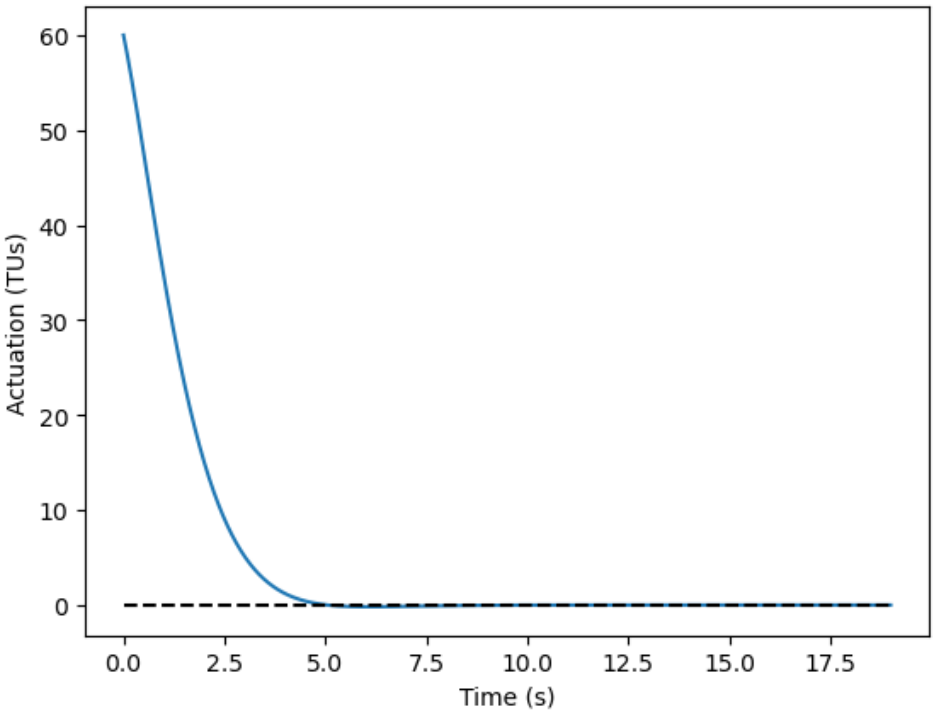


Figure 3. Displacement(u) Vs. Time (s) Model Response

Table 2.1 Response characteristics vs. Kp, with Kd = 0.

Proportional gain Kp	Percentage overshoot %OS	Settling Time ts
400	0	100
410	0	40
420	0	25
430	0	16
440	0.0120	17
450	0.0311	17.5

Table 2.2 Response characteristics vs. Kd, with Kp = 600.

Derivative gain Kd	Percentage overshoot %OS	Settling Time ts
0	0.263	15
50	0.152	12.5
100	0.075	12
150	0.026	10
200	0.003	6.5

Discussion

What are the effects of Kp and Kd on transient response? Justify your answer.

Kp effects the transient response specifically by speeding up response. When Kp increases, the settling time dramatically decreases, at the cost of introducing overshoot. As shown in Table 2.1, settling time decreases from 100 to 17.5 seconds, but overshoot of .03 is introduced.

When Kd increases, it also minorly decreases settling time, but mainly decreases percentage of overshoot. This hones in on decreasing error. Table 2.2 shows this, where the percent overshoot decreases from .26 to .003 when Kd is increased from 0 to 200.

Appendix A

In [1]:
```python
def Fibonacci(n):
    # Requirement 2: handle negative input
    if n < 0:
        print("Incorrect input.")
        return [], 0

    # Requirement 3: handle n = 0 and n = 1
    if n == 0:
        F = []
        return F, 0

    if n == 1:
        F = [0]
        return F, 0

    # Requirement 4: initialize list
    F = [0, 1]

    # Generate sequence using for loop
    for i in range(n - 2):
        F_new = F[i+1] + F[i]
        F.append(F_new)

    # Requirement 5: compute sum
    sum_F = sum(F)

    return F, sum_F
```

In [2]:
```python
print(Fibonacci(-1))   # incorrect input
print(Fibonacci(0))    # edge case
print(Fibonacci(1))    # edge case
print(Fibonacci(5))    # normal case
print(Fibonacci(10))   # larger case
```

```
Incorrect input.
([], 0)
([], 0)
([0], 0)
([0, 1, 1, 2, 3], 7)
([0, 1, 1, 2, 3, 5, 8, 13, 21, 34], 88)
```

Figure A.1. Section 2.3.3 showing Python Warmup assignment. Function “Fibonacci” accepts one input *n*, and outputs the first *n*-th Fibonacci number and their sum.